

Mobility Diary: un sistema Context-Aware per il diario personale di mobilità con Human Activity Recognition e privacy sulla posizione

Giacomo Bianco

Dipartimento di Informatica – Scienza e Ingegneria

Università di Bologna

Bologna, Italia

giacomo.bianco4@studio.unibo.it

Abstract—

Index Terms—context-awareness, human activity recognition, location privacy, mobility diary

I. INTRODUZIONE

Comprendere come le persone si muovono all'interno di un contesto urbano è un problema di crescente interesse, con applicazioni che spaziano dalla pianificazione dei trasporti alla progettazione di servizi personalizzati per il singolo utente. Il semplice tracciamento della posizione, tuttavia, non è sufficiente a cogliere il significato degli spostamenti: una sequenza di coordinate GPS non distingue, ad esempio, tra un tragitto percorso a piedi, in bicicletta o in automobile, né indica se un punto visitato rappresenti un luogo rilevante per l'utente, come la propria abitazione o il luogo di lavoro. Un sistema *context-aware* per la mobilità deve quindi andare oltre la semplice acquisizione di dati grezzi di posizione, arricchendo la traccia con altre informazioni, quali l'attività di movimento riconosciuta e i luoghi significativi frequentati, in modo da trasformare una sequenza di punti GPS in un diario di mobilità realmente informativo.

In questo lavoro presento Mobility Diary, un sistema context-aware composto da un'applicazione mobile per la raccolta dei dati di mobilità, un back-end che ne gestisce l'elaborazione e la persistenza, e una dashboard Web per la consultazione del diario. Il sistema integra il riconoscimento dell'attività umana (Human Activity Recognition, HAR) con meccanismi di protezione della privacy sui dati di posizione. Nello specifico, il sistema realizza:

- una pipeline di ingestione asincrona dei dati grezzi raccolti dal dispositivo mobile, basata su un pattern di caricamento diretto verso uno storage a oggetti (S3-compatible) seguito da una fase di conferma esplicita (*direct-to-storage upload with confirm step*), che evita che i dati grezzi transitino dal server applicativo, seguito da un'elaborazione asincrona lato server che materializza la traccia di mobilità;
- la ricostruzione della mobilità dell'utente come sequenza di segmenti temporali, ciascuno associato a un luogo o

a un'attività di movimento (cammino, bicicletta, automobile, sosta), a partire dalle tracce raccolte dal dispositivo;

- un modulo di Human Activity Recognition basato su una pipeline a due fasi CNN1D + GRU bidirezionale, addestrata sul dataset pubblico SHL (Sussex-Huawei Locomotion) per riconoscere cinque modalità di mobilità (*idle, walking, running, biking, moving vehicle*), con un'accuratezza dell'84,9% sul validation set ufficiale dopo l'ottimizzazione del modello per l'inferenza in produzione;
- la generazione di viste privacy-aware del diario a tre livelli (*precise, approximate, aggregated*), ottenute tramite una tecnica di generalizzazione spaziale a griglia (*spatial cloaking*) sulle tracce GPS, con valutazione quantitativa del compromesso tra perturbazione della posizione e qualità del servizio (Privacy Perturbation e Quality of Service).

Il resto dell'articolo è organizzato come segue: la Sezione II descrive l'architettura del sistema, la Sezione III ne dettaglia l'implementazione e la Sezione IV presenta i risultati sperimentali ottenuti.

II. ARCHITETTURA DEL SISTEMA

A. Panoramica

Il sistema è composto da due client – l'applicazione mobile e la dashboard Web – e da un back-end raggiungibile attraverso un unico gateway, come mostrato in Fig. 1. Il back-end gestisce sia le operazioni sincrone del diario, sia l'elaborazione asincrona dei dati grezzi caricati dal dispositivo, descritte nel dettaglio nelle sottosezioni seguenti. Il dato nasce sul dispositivo dell'utente, viene caricato verso il back-end e sottoposto a un'elaborazione asincrona che ne estrae informazioni di contesto (attività riconosciuta, luoghi significativi); viene infine persistito e reso disponibile in lettura, con o senza protezione della privacy, alla dashboard Web.

B. Applicazione mobile

L'applicazione mobile, sviluppata in Flutter, è responsabile della raccolta dei dati sul dispositivo dell'utente. Localmente,

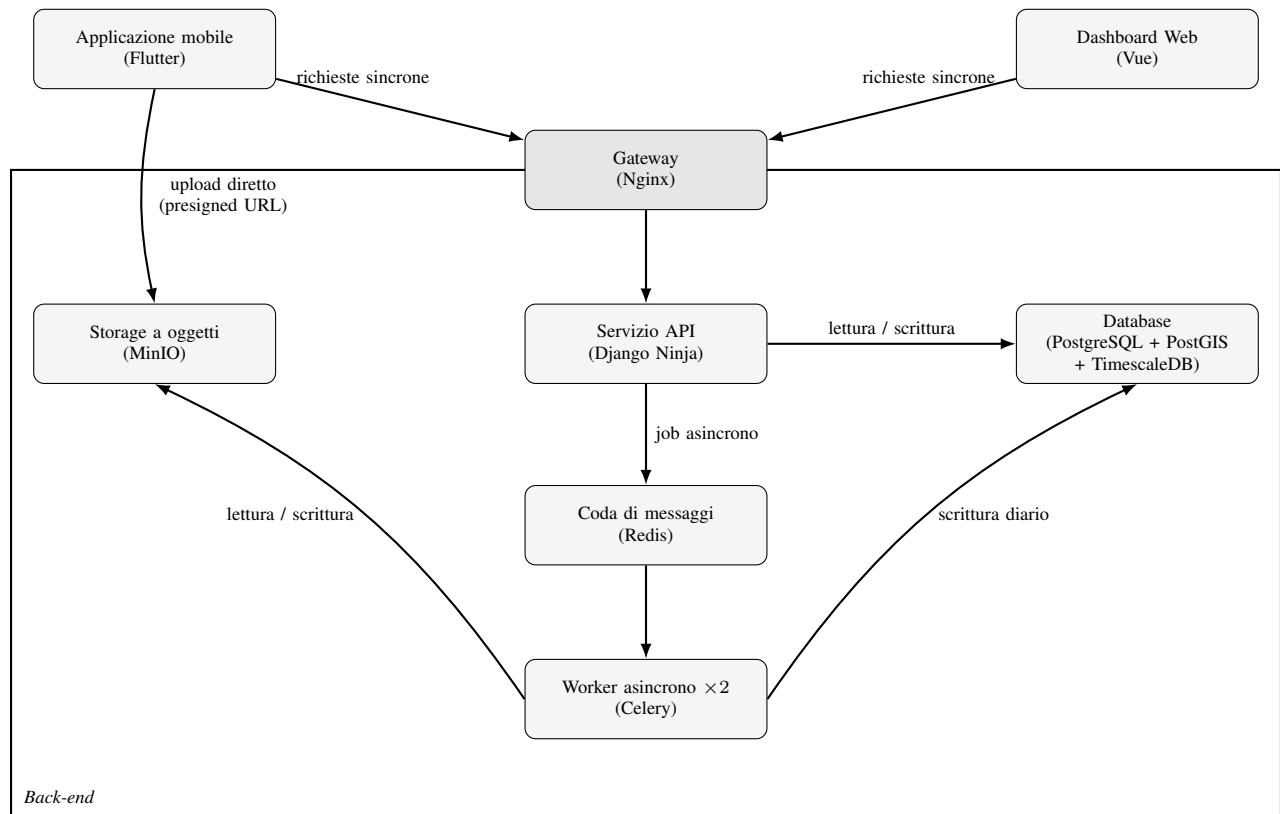


Fig. 1. Architettura ad alto livello del sistema.

l'applicazione determina in autonomia se l'utente è in movimento o fermo, tramite una macchina a stati finiti alimentata dai dati dei sensori, in modo da adattare la frequenza di campionamento in base allo stato ed evitare un'acquisizione continua e costante che porterebbe ad un consumo eccessivo della batteria. I dettagli di questa macchina a stati sono discussi nella Sezione III.

C. Livello di ingresso e servizio applicativo

Le richieste sincrone dell'applicazione mobile e della dashboard attraversano un'unica istanza di gateway (Nginx), che instrada il traffico verso il servizio applicativo, realizzato in Django con Django Ninja e replicato su due istanze per poter servire richieste concorrenti. Questo livello espone le principali operazioni sincrone del sistema: autenticazione dell'applicazione mobile e della dashboard, lettura del diario giornaliero e delle tracce di mobilità, gestione dei luoghi significativi (conferma, rifiuto, etichettatura), consultazione delle statistiche personali, gestione delle preferenze di privacy ed esportazione privacy-aware del diario, oltre alla gestione degli utenti lato dashboard. È inoltre responsabile di schedare nella coda di messaggi i task relativi alle operazioni che richiedono un'elaborazione asincrona.

D. Pipeline di ingestione ed elaborazione

Il caricamento dei dati grezzi raccolti dal dispositivo non avviene attraverso il servizio applicativo, ma tramite un pattern

di caricamento diretto verso uno storage a oggetti, seguito da una fase di conferma esplicita verso il back-end. Questa scelta evita che il servizio applicativo non debba occuparsi del trasferimento di dati che potrebbero occupare a lungo le sue connessioni, saturandone la capacità di servire richieste concorrenti: il suo ruolo si limita ad autorizzare il caricamento e a confermarne il completamento, mentre il trasferimento dei dati avviene direttamente tra dispositivo e storage.

Una volta confermato il caricamento, il dato viene preso in carico dal livello di elaborazione asincrona (Celery), disaccoppiato dal servizio applicativo tramite una coda di messaggi (Redis). L'elaborazione avviene in quattro fasi in sequenza, ciascuna dipendente dall'output della precedente:

- 1) **classificazione dell'attività**: che etichetta le finestre di dati sensoriali presenti nello storage con la modalità di movimento riconosciuta;
- 2) **inserimento nel diario**: che materializza la traccia di mobilità, i relativi segmenti e le etichette di movimento a partire dai dati caricati e dalle etichette prodotte al passo precedente;
- 3) **inserimento batch dei dati sensoriali**: che persiste le misure grezze nel database (TimescaleDB);
- 4) **analisi dei luoghi abituali**: che individua i luoghi significativi frequentati dall'utente a partire dai dati di posizione inseriti nel diario.

La natura asincrona di queste quattro fasi permette di gestire carichi di lavoro computazionalmente costosi (in particolare

fase 1 e fase 3) senza impattare la latenza delle richieste sincrone del servizio applicativo. I dettagli realizzativi di ciascuna fase sono discussi nella Sezione III.

E. Persistenza

Il sistema utilizza due meccanismi di persistenza distinti, scelti in base alla natura del dato. I dati strutturati del diario, ovvero tracce, segmenti, luoghi significativi ed etichette di attività, sono memorizzati in un database relazionale esteso con supporto per dati spaziali e per serie temporali, che permettono di eseguire query geografiche e query di analisi pesanti basate su aggregazioni. I dati grezzi provenienti dai sensori, che non necessitano di essere interrogati singolarmente ma solo recuperati per intero durante l'elaborazione asincrona, sono invece mantenuti in uno storage a oggetti separato dal database.

F. Dashboard

La dashboard Web, realizzata in Vue, non è pensata per l'utente finale ma per un amministratore: consente di selezionare un utente da un elenco e di ispezionarne il diario di mobilità a fini di supporto e verifica. Consuma le API sincrone esposte dal servizio applicativo per mostrare, per l'utente selezionato, le tracce di mobilità e i luoghi significativi su mappa, il diario giornaliero in forma tabellare e temporale, e le statistiche aggregate sulle attività riconosciute prodotte dalla fase di data mining della pipeline. Permette inoltre di confrontare la traccia reale con la corrispondente vista privacy-aware, e di filtrare i dati mostrati per giorno, attività riconosciuta, luogo significativo o intervallo temporale.

III. IMPLEMENTAZIONE

A. Applicazione mobile

1) Macchina a stati finiti di rilevamento del movimento:

L'applicazione è pensata per restare in esecuzione anche per diverse ore nell'arco della giornata, il che rende il consumo energetico un vincolo di progettazione non trascurabile. Durante l'intero utilizzo, il sistema mantiene attivi tre stream di sensori: accelerometro, giroscopio e GPS. Per contenere il consumo energetico senza rinunciare alla qualità del dato, l'applicazione mantiene localmente una macchina a stati finiti che determina se l'utente è in movimento (*movement*) o fermo (*stationary*) (Sezione II), e associa a ciascuno dei due stati un profilo di campionamento diverso, riducendo la frequenza di acquisizione quando l'utente è fermo.

Per decidere se cambiare stato, la macchina osserva due valori: la velocità istantanea fornita dal GPS e σ , la deviazione standard della Signal Vector Magnitude (SVM) dell'accelerometro – la SVM di un campione è $\sqrt{x^2 + y^2 + z^2}$ sulle tre componenti dell'accelerazione. σ non viene ricalcolato a ogni singolo campione dell'accelerometro, ma accumulando i campioni in una finestra e calcolandolo una sola volta al suo completamento: 20 campioni (2 s a 10 Hz) nello stato *stationary*, 500 campioni (5 s a 100 Hz, la stessa finestra usata per la classificazione HAR) nello stato *movement*.

All'avvio di un viaggio la macchina viene inizializzata nello stato *stationary*. In questo stato il profilo di campionamento imposta l'accelerometro a 10 Hz e disattiva il giroscopio (attivo invece a 100 Hz in *movement*, dove alimenta le finestre sensoriali per la classificazione HAR); il GPS, in entrambi gli stati, non campiona a una frequenza fissa ma emette un nuovo fix quando l'utente si sposta di almeno 3 m dall'ultima posizione nota. Una soglia di spostamento più piccola produrrebbe un numero di fix eccessivo rispetto alla precisione del GPS, che finirebbero comunque tutti persistiti nel database, aumentando il volume di dati archiviati senza un beneficio proporzionale sulla qualità della traccia. La Tabella I riassume la frequenza dei tre sensori nei due stati, e dopo quanti dati ciascuno produce l'evento che fa scattare una nuova valutazione della macchina a stati, cioè un nuovo calcolo dell'evidenza (movimento, sosta o incerta) a partire dal dato appena arrivato.

TABLE I
FREQUENZA DEI SENSORI E GENERAZIONE DEGLI EVENTI PER LA
MACCHINA A STATI

Sensore	station.	movem.	Innesca una valutazione dopo
Accelerometro	10 Hz	100 Hz	20 campioni (2s) in <i>stationary</i> ; 500 in <i>movement</i> (5s)
Giroscopio	spento	100 Hz	non alimenta la FSM: solo finestre HAR
GPS	a evento (3 m)	a evento (3 m)	immediato, a ogni fix

Accelerometro e GPS sono quindi due processi indipendenti e asincroni: ogni volta che uno dei due produce un nuovo dato, la macchina rivaluta lo stato corrente combinando quel dato con l'ultimo valore noto dell'altro segnale, entrambi soggetti a un controllo di freschezza (rispettivamente 10 s e 20 s) che considera assente un dato più vecchio della soglia. Il risultato di questa valutazione è un'evidenza, movimento, sosta o incerta, ottenuta confrontando σ e la velocità GPS con soglie fisse.

Le soglie non sono le stesse in ogni situazione. Una velocità GPS pari o superiore a 2.5 m/s è considerata da sola sufficiente a dichiarare evidenza di movimento, indipendentemente da σ , poiché compatibile solo con un veicolo. Al di sotto di questa soglia, se lo stato corrente è *movement* la decisione si basa solo su σ : valori pari o superiori a 1.2 indicano movimento, valori inferiori a 0.8 indicano sosta, i valori intermedi restano incerti. Viene data priorità a σ per due motivi legati fra loro: quando la macchina a stati si trova in *movement* e deve stabilire se l'utente si è davvero fermato, la sosta avviene tipicamente in un luogo chiuso, dove il GPS diventa meno preciso; inoltre, se l'utente è realmente fermo, il filtro di distanza di 3 m descritto sopra fa sì che il GPS smetta proprio di inviare nuovi dati. Il segnale GPS quindi non solo perde precisione ma smette di fornire nuova informazione proprio nel momento in cui servirebbe per confermare la sosta, mentre l'accelerometro resta affidabile. Se invece lo stato corrente è *stationary*, la macchina a stati richiede che σ e la velocità GPS concordino: evidenza di movimento solo se $\sigma \geq 1.2$ e velocità ≥ 0.8 m/s,

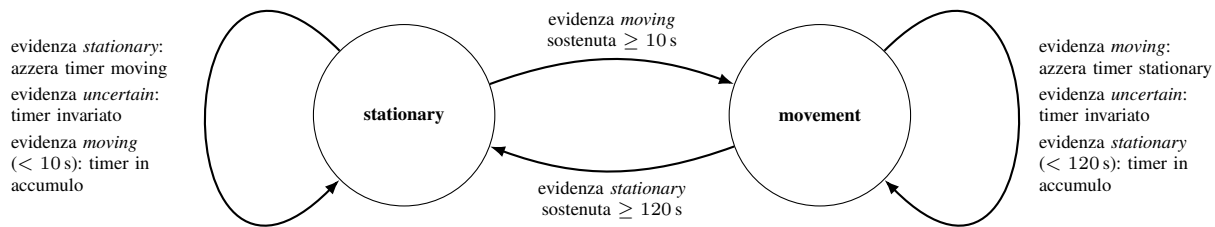


Fig. 2. Macchina a stati finiti di rilevamento del movimento: stati, transizioni ed effetto di ciascun tipo di evidenza sui timer di isteresi.

evidenza di sosta solo se $\sigma < 0.8$ e velocità < 0.5 m/s; in tutti gli altri casi l'evidenza resta incerta. Questo requisito più stringente nello stato `stationary` riduce i falsi positivi che un singolo segnale rumoroso potrebbe generare quando l'utente è effettivamente fermo: il sistema è pensato proprio per non segnalare come movimento i piccoli spostamenti tipici di un ambiente chiuso, come pochi passi in casa o in palestra. A questo si aggiunge un vincolo temporale: le transizioni di stato sono soggette a un tempo di conferma minimo asimmetrico, che richiede un'evidenza sostenuta per un tempo minimo prima di confermare il cambio di stato. Questo è un ulteriore filtro contro i micro-spostamenti, che raramente durano abbastanza a lungo da superarla. Da `stationary` a `movement` sono richiesti 10 s di evidenza di movimento sostenuta, mentre da `movement` a `stationary` ne sono richiesti 120 s. Questa asimmetria evita che una breve sosta durante uno spostamento (ad esempio a un semaforo) chiuda prematuramente un segmento di movimento. Ad ogni transizione, la macchina aggiorna il profilo di campionamento dei sensori. Quando lo stato diventa `movement`, il cambiamento è drastico: il filtro del GPS resta invariato, ma il giroscopio si accende e accelerometro e giroscopio passano entrambi a 100 Hz. Questa frequenza non è arbitraria, ma coincide con quella dei dati del dataset SHL usato per addestrare il modello di Human Activity Recognition (Sezione III), poiché le finestre sensoriali raccolte in questo stato sono quelle poi effettivamente classificate dal modello. La Fig. 2 riassume gli stati, le transizioni e tutti i casi di evidenza gestiti dalla macchina.

2) *Rapporto tra macchina a stati e sistema HAR:* La macchina a stati e il sistema di Human Activity Recognition non hanno lo stesso peso nella costruzione del diario: la macchina a stati è la prima fonte di verità. Un segmento temporale che la macchina ha classificato come `stationary` viene considerato sosta ed è preso per buono come tale nel diario finale; il sistema HAR non interviene su questi segmenti. Il suo ruolo si limita a classificare la modalità di movimento

all'interno dei soli intervalli di tempo che la macchina a stati ha già decretato come `movement`.

Questa gerarchia si riflette direttamente su cosa viene persistito localmente sul dispositivo. Le finestre di dati di accelerometro e giroscopio, destinate alla classificazione HAR, vengono scritte nel database locale (SQLite) solo durante le fasi in cui la macchina a stati ha dichiarato `movement`: quando lo stato è `stationary` i dati dei sensori (solo l'accelerometro, poiché il giroscopio è spento) vengono usati esclusivamente per la decisione della macchina stessa e non generano alcuna finestra da salvare. Questo non esclude che, all'interno di una finestra classificata come `movement`, il sistema HAR possa poi riconoscere sotto-intervalli in cui l'utente è di fatto fermo (ad esempio una breve sosta a un semaforo); questo caso è discusso più avanti (Sezione III).

I dati del GPS seguono invece una logica diversa: vengono registrati per tutta la durata del viaggio, incluse le fasi di sosta. Vengono comunque sempre persistiti perché sono necessari a ricostruire il viaggio nella sua interezza, anche dal punto di vista grafico sulla mappa. La Fig. 3 riassume, lungo un viaggio composto da tre segmenti consecutivi, quali dati vengono effettivamente persistiti localmente in SQLite in ciascuna fase, prima di essere caricati e incorporati dal back-end secondo la pipeline di ingestione descritta in Sezione II-D.

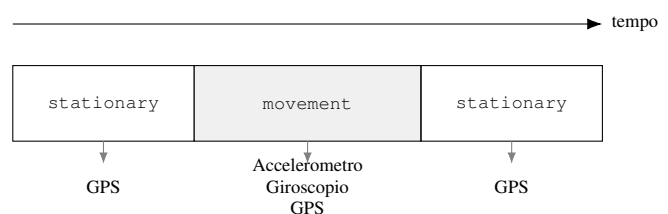


Fig. 3. Dati persistiti localmente in SQLite lungo un viaggio composto da tre segmenti: nelle fasi `stationary` viene salvato solo il GPS, mentre nella fase `movement` vengono salvati anche accelerometro e giroscopio, usati per la classificazione HAR.

3) *Struttura e impacchettamento dei dati sensoriali per l'HAR*: Come descritto in Sezione II, nello stato *movement* accelerometro e giroscopio campionano entrambi a 100 Hz. Ogni 5 s di campioni viene raggruppato in una finestra, ovvero una matrice 500×6 : 500 campioni temporali per le sei componenti *Acc_x*, *Acc_y*, *Acc_z*, *Gyr_x*, *Gyr_y*, *Gyr_z*. Localmente, ogni finestra è persistita come una singola riga della tabella *SensorWindows* del database SQLite: la matrice è serializzata in formato JSON nella colonna *matrixJson*, accompagnata da metadati (timestamp di inizio e fine, numero di campioni, frequenza).

Al momento della sincronizzazione, le finestre non vengono caricate una alla volta né tutte insieme, ma raggruppate in più file. Le righe della tabella vengono lette dal database a pagine di 128 finestre e accumulate in un buffer finché la loro dimensione, misurata sul JSON non compresso, non supera un budget di 12 MB; a quel punto il buffer viene scritto su un file e se ne apre uno nuovo. Ogni finestra pesa circa 29 KB da non compressa: il JSON è testo, e ogni valore viene prima arrotondato a sei decimali, ad esempio per una singola riga a sei canali:

```
[−0.023457, −9.806649, 0.112346,  
0.001235, −0.004321, 0.000765]
```

Con valori di questa forma, una riga pesa circa 58 caratteri (51 per i sei numeri più 7 tra virgole e parentesi), pari a circa 58 byte in UTF-8, dove ogni cifra o simbolo occupa esattamente un byte; su 500 righe per finestra si ottengono circa 29 000 byte, coerenti con il valore osservato. Un file da 12 MB contiene quindi approssimativamente 430 finestre (circa 36 minuti di registrazione). Questo budget è tenuto volutamente basso per limitare il picco di memoria della serializzazione sul telefono, indipendentemente dalla durata complessiva del viaggio.

Ogni file viene poi compresso con GZIP, un formato di compressione senza perdita basato sull'algoritmo DEFLATE: riduce la dimensione del file individuando e codificando in modo più compatto le ripetizioni nei dati, il che si adatta bene a un JSON con una struttura molto regolare come questo, dimezzandone circa la dimensione (i 12 MB non compressi diventano circa 5 MB). L'intera operazione, ovvero compressione e calcolo del checksum SHA-256, viene eseguita in un isolate separato dall'interfaccia utente, il meccanismo di Flutter per eseguire codice su un thread indipendente, questo evita che un calcolo pesante come questo possa bloccare il rendering dell'interfaccia. Il checksum calcolato serve alla successiva fase di conferma dell'upload (Sezione III): la stringa SHA-256 calcolata localmente viene inviata al back-end insieme al file, che la confronta con il checksum calcolato sul file effettivamente ricevuto sullo storage, per verificare che corrispondano.

4) *Coda di sincronizzazione offline*: I dati raccolti localmente vengono sincronizzati verso il back-end attraverso la tabella *SyncJobs* del database SQLite locale, un record della quale gestisce di fatto una coda di sincronizzazione: ogni sessione di raccolta genera un job, elaborato in ordine

cronologico – i più vecchi per primi – uno alla volta, per evitare di saturare la connessione con più sessioni in parallelo.

Il caricamento di ogni job avviene in due fasi. Nella fase *core*, i punti GPS raccolti e i cambiamenti di stato tra *movement* e *stationary* vengono inviati con una richiesta diretta al servizio API, senza passare per lo storage a oggetti. Nella fase *raw*, vengono invece caricati i file prodotti come descritto nella sottosezione precedente, secondo il pattern di caricamento diretto con conferma descritto in Sezione II-D: ciascuna parte viene prima autorizzata dal back-end, poi caricata e infine confermata con il relativo checksum. Ogni job traccia separatamente lo stato delle due fasi, che possono essere confermate e ritentate in modo indipendente; le parti *raw* vengono caricate a gruppi di tre in parallelo, anziché una alla volta, per ridurre il tempo complessivo di sincronizzazione.

La coda gestisce esplicitamente i casi in cui il back-end sta ancora elaborando in modo asincrono i dati ricevuti, mettendo il job in attesa e ripetendo il controllo dello stato a intervalli regolari. In caso di errore, il job viene ritentato con un ritardo crescente (10 s, 30 s, 2 min, 10 min) fino a un massimo di cinque tentativi, oltre i quali viene scartato; un job scartato per cui il back-end ha già creato una traccia viene inoltre annullato lato server, per non lasciare dati parziali orfani. Al completamento con successo, i file temporanei locali vengono eliminati e la sessione viene rimossa dal database locale, mantenendo limitato lo spazio occupato sul dispositivo.

5) *Vincolo di un viaggio attivo alla volta*: Il sistema è pensato attorno a un'unica linea temporale per utente: il diario è una sequenza di viaggi consecutivi, e non ha senso, in questo modello, che due viaggi risultino in corso nello stesso momento. Questo vincolo è realizzato facendo sì che, nel momento in cui l'utente preme Start, venga già creato lato back-end un record del viaggio – con data di inizio e dispositivo associato – anche se in quel momento i dati non vengono ancora inviati: l'invio effettivo di tracce GPS e dati sensoriali non è progressivo durante la registrazione, ma avviene tutto in blocco solo alla pressione di Stop, secondo la coda di sincronizzazione descritta in precedenza.

L'esistenza di questo record fin dall'avvio è ciò che permette di far rispettare il vincolo: se l'utente prova ad avviare un nuovo viaggio da un dispositivo diverso mentre, sullo stesso account, un viaggio risulta già avviato e non ancora concluso, il back-end rifiuta la richiesta con un conflitto, impedendo che due registrazioni si sovrappongano sulla stessa linea temporale.

Durante il viaggio, l'applicazione invia inoltre al back-end un segnale di *heartbeat* ogni 5 minuti, che aggiorna un dato di freschezza associato al viaggio in corso. Questo meccanismo protegge da un caso limite: un viaggio che risulti ancora attivo ma che in realtà non riceva aggiornamenti da troppo tempo. Se un viaggio attivo resta senza heartbeat per più di 24 ore, viene chiuso automaticamente non appena l'utente prova ad avviarne uno nuovo – da un altro dispositivo o dallo stesso – permettendo alla registrazione di ripartire anziché restare bloccata indefinitamente da un viaggio ormai inattivo.

6) *Visualizzazione progressiva dei risultati*: Riprendendo le due fasi di caricamento (*core* e *raw*) e le quattro fasi asincrone di elaborazione descritte in Sezione II-D, l'applicazione mobile non attende il completamento dell'intera pipeline per mostrare all'utente il viaggio appena registrato. Non appena la fase *core* è confermata, l'applicazione mostra subito la mappa del viaggio nella sua interezza, costruita a partire dai punti GPS caricati in quella fase: è l'informazione più immediata da mostrare, ed è già disponibile lato back-end a quel punto.

Per ottenere il diario con i segmenti colorati in base alla modalità di movimento, l'applicazione effettua invece un polling verso il back-end con cadenza di un secondo, richiedendo a ogni chiamata lo stato di elaborazione del viaggio. Il back-end restituisce il diario come completo solo quando il viaggio risulta processato, condizione che si verifica nel momento in cui le fasi 1 e 2 della pipeline asincrona – classificazione dell'attività e inserimento nel diario – sono terminate; le fasi 3 e 4 – inserimento batch dei dati sensoriali e data mining dei luoghi abituali – non sono invece necessarie per mostrare il diario e proseguono indipendentemente. Il punto rilevante è quindi che l'utente non deve attendere il completamento dell'intera elaborazione asincrona per vedere, graficamente, l'esito del proprio viaggio, ma solo il termine delle prime due fasi: questo riduce sensibilmente il tempo di caricamento percepito rispetto all'attesa dell'intera pipeline. La Fig. 4 riassume l'intera pipeline, dalla pressione di Stop fino al completamento del data mining, insieme ai due momenti in cui l'utente vede effettivamente qualcosa cambiare nell'interfaccia.

7) *Gestione dell'applicazione in background*:

L'applicazione è pensata per un utilizzo prolungato nell'arco della giornata: l'utente preme Start e lascia l'app in esecuzione, spesso in background, per diverse ore. In questa condizione l'applicazione deve gestire con attenzione sia il caso in cui il sistema operativo la sospenda, sia più semplicemente il caso in cui l'utente non vi torni per diversi minuti. La continuità della raccolta dati in questi stati non è garantita allo stesso modo per tutti i sensori: i permessi di localizzazione in background permettono di continuare a ricevere fix GPS, ma le librerie usate per accelerometro e giroscopio non possono garantire che i campioni continuino ad arrivare in ogni condizione di background del sistema operativo.

Per questo motivo, quando l'utente torna nell'applicazione dopo un'assenza, il sistema confronta i timestamp dell'ultima transizione di stato, dell'ultimo punto GPS e dell'ultima finestra sensoriale registrati localmente, e prende il più recente dei tre come istante dell'ultima attività nota. Se da quell'istante sono trascorsi almeno 30 minuti, la sessione viene considerata autoconclusa: viene chiusa retroattivamente, con istante di fine impostato a quell'ultima attività nota anziché al momento della riapertura, e inviata al back-end come se il viaggio si fosse concluso in quel momento. Se invece sono trascorsi meno di 30 minuti, la sessione viene ripresa normalmente. In questo caso, tuttavia, il sistema effettua una correzione: se l'ultimo stato noto della macchina era *movement*, ma nel frattempo

i dati dei sensori hanno smesso di arrivare, quel *movement* non è più considerato affidabile – non essendoci conferma recente che l'utente sia ancora in movimento – e il sistema forza lo stato a *stationary* a partire da quando i dati si sono interrotti; la macchina a stati riprende quindi da questo stato corretto, anziché dall'ultimo realmente osservato.

Questo meccanismo lato mobile, però, non basta da solo: può comunque capitare che un segmento venga classificato dalla macchina a stati come *movement*, ma che i dati sensoriali corrispondenti a quell'intervallo non siano di fatto disponibili. Per questo esiste un'ulteriore rete di sicurezza lato back-end: quando, durante la costruzione del diario, un segmento di movimento non ha finestre HAR classificate al suo interno, il sistema non lo lascia privo di etichetta, ma stima l'attività a partire dalla velocità mediana registrata dai punti GPS in quell'intervallo, classificandola secondo soglie fisse (a piedi sotto gli 8 km/h, di corsa fino a 13 km/h, in bicicletta fino a 25 km/h, altrimenti in veicolo).

B. *Classificazione dell'attività (HAR)*

1) *Dataset e preparazione*: Il modulo HAR è addestrato sul dataset pubblico SHL (Sussex-Huawei Locomotion), raccolto dall'Università del Sussex con smartphone Huawei indossati simultaneamente in quattro posizioni del corpo (tasca, borsa, petto, mano). Ogni posizione registra a 100 Hz accelerometro, giroscopio e magnetometro, già pre-segmentati in finestre da 500 campioni (5 s). Mescolare le quattro posizioni nel training rende il modello robusto al fatto che, in un uso reale, non si sa dove l'utente tenga il telefono.

Le otto classi originali del dataset (IDLE, WALKING, RUNNING, DRIVING, BIKING, BUS, TRAIN, SUBWAY) sono state ridotte a cinque, fondendo le quattro modalità di trasporto motorizzato in un'unica classe *MOVING_VEHICLE*: le confusioni tra queste modalità erano numerose (le firme inerziali di un'auto e di un mezzo pubblico sono molto simili) e la fusione le trasforma da errori in predizioni corrette, senza eliminare informazione utile al sistema, che non deve distinguere tra mezzi di trasporto motorizzati diversi.

Una prima valutazione con split casuale del dataset (70/15/15) dava un'accuratezza dell'84,0%. Le finestre consecutive distano però solo 5 s e sono quasi identiche tra loro: uno split casuale lascia quindi finestre praticamente uguali sia nel training sia nel test, gonfiandone artificialmente l'accuratezza (*data leakage* temporale). Valutando lo stesso modello sul validation set ufficiale di SHL – una sessione di registrazione mai vista in training – l'accuratezza onesta scende al 73,3%, undici punti percentuali più bassa. Tutte le valutazioni successive, comprese quelle riportate qui, usano esclusivamente questo protocollo corretto: training set ufficiale per l'addestramento, l'ultima porzione temporale del training per l'early stopping, e il validation set ufficiale solo per la valutazione finale.

2) *Architettura del modello*: Il classificatore di base è una CNN1D che scorre lungo il tempo, leggendo tutti i canali dei sensori a ogni passo: due blocchi convoluzione+pooling (Conv1D(64), Conv1D(128)), kernel

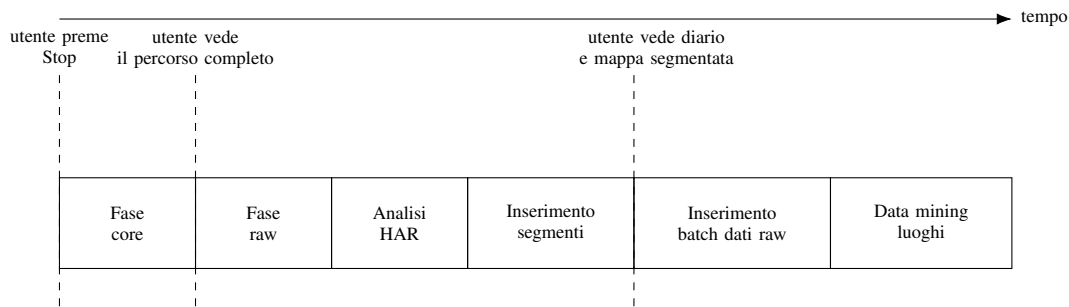


Fig. 4. Pipeline completa di caricamento ed elaborazione.

10, con `MaxPooling1D`) seguiti da un `Dense(128)` e da uno strato di uscita softmax sulle cinque classi, per circa 2 milioni di parametri complessivi.

A partire da questa base, il modello finale è stato raggiunto per passi incrementali, ciascuno motivato da un problema osservato sul validation set ufficiale:

- uno *smoothing* temporale a maggioranza in post-processing, che sfrutta il fatto che le attività hanno inerzia (non si cambia stato ogni 5 s), ha recuperato 7,2 punti (dal 73,3% all'80,5%) senza riaddestrare nulla;
- l'aggiunta del magnetometro (9 canali), una normalizzazione per-canale calcolata solo sul training set e una *focal loss* per contrastare lo sbilanciamento delle classi (`MOVING_VEHICLE` è circa il 51% dei dati) hanno portato il modello grezzo al 75,4%, all'80,8% con lo smoothing;
- sostituire lo smoothing, una regola fissa, con un contesto temporale appreso – una GRU bidirezionale che legge sequenze di *embedding* prodotti dalla CNN su ciascuna finestra – ha infine portato l'accuratezza onesta all'83,7%, il risultato migliore ottenuto.

L'errore residuo dominante, anche nel modello migliore, resta la confusione tra `IDLE` e `MOVING_VEHICLE`: un veicolo fermo a lungo (semaforo, ingorgo, stazione) produce un segnale inerziale indistinguibile da quello di una persona ferma. È un limite fisico dei soli sensori inerziali, che né il magnetometro né il contesto temporale risolvono del tutto, e che rappresenta la maggioranza degli errori residui del classificatore.

3) *Ottimizzazione per l'inferenza in produzione*: Il modello effettivamente distribuito in produzione non è quello a 9 canali appena descritto, ma una variante a 6 canali (solo accelerometro e giroscopio): il magnetometro dei telefoni reali si è rivelato meno affidabile di quello del dataset SHL, tanto da rendere preferibile ometterlo. La sua accuratezza onesta sul validation ufficiale è del 79,47%.

Nella pipeline a due fasi originale (CNN e GRU come due modelli separati), l'inferenza in produzione mostrava tempi incoerenti e non proporzionali al numero di finestre elaborate. L'analisi ha individuato più cause concorrenti: la ricompilazione del grafo di calcolo (*graph retracing*) ogni volta che `model.predict()` riceveva una dimensione di batch mai vista dal processo, costosa circa 2,3 s per la rete

CNN+GRU; un ciclo Python che chiamava la GRU una volta per ogni gruppo di 32 finestre invece che con un'unica chiamata sull'intero batch, circa 4,4 volte più lento; e uno stato dell'ottimizzatore Adam rimasto nei checkpoint, inutile in inferenza e responsabile di gran parte della loro dimensione su disco.

La soluzione applicata combina due interventi indipendenti. Primo, la CNN e la GRU sono state fuse in un unico modello end-to-end, con i pesi copiati (non riaddestrati) dai due checkpoint esistenti e l'equivalenza numerica verificata rispetto alla pipeline originale (differenza massima sulle probabilità dell'ordine di 10^{-7} , rumore numerico). Secondo, il modello fuso è avvolto in una `tf.function` con una `input_signature` esplicita a batch dinamico ma lunghezza di sequenza fissa, così da forzare un'unica compilazione valida per qualunque numero di finestre per richiesta, eseguita una volta sola al posto di ripetersi a ogni nuova dimensione di batch; una chiamata di riscaldamento all'avvio del processo worker sposta questo costo di compilazione dalla prima richiesta reale al momento dell'inizializzazione.

In questa stessa fase è emerso che la lunghezza di sequenza della GRU, fissata a 32 finestre nella pipeline originale, non era mai stata ritarata dopo il passaggio a 6 canali: aumentarla a 128 finestre ha migliorato l'accuratezza onesta dal 79,47% all'84,90%, un compromesso scelto perché vicino al plateau del guadagno (che continua solo marginalmente oltre le 256 finestre) a un costo di latenza ancora trascurabile. Il guadagno viene soprattutto dalla confusione dominante `IDLE/MOVING_VEHICLE`, a fronte di una lieve perdita di recall sulla classe `BIKING`.

Misurato sullo stesso processo e sugli stessi dati, il confronto tra la pipeline originale e quella ottimizzata mostra un miglioramento sia di latenza sia di accuratezza, indipendenti tra loro: il primo deriva dalla correzione del pattern di chiamata (a rischio nullo, le predizioni restano identiche a parità di lunghezza di sequenza), il secondo dall'aumento del contesto temporale della GRU.

Sull'intero validation set ufficiale (115.156 finestre), l'accuratezza sale dal 79,48% all'84,88% e il tempo totale di elaborazione scende da 68,4 s a 25,7 s (2,7 volte più veloce in throughput aggregato).

TABLE II
LATENZA DELL'INFERENZA, PIPELINE ORIGINALE VS OTTIMIZZATA
(STESSO PROCESSO, A CALDO)

Finestre	Originale	Ottimizzata	Speedup
141	110,9 ms	42,1 ms	2,6×
579	410,6 ms	93,6 ms	4,4×
1060	728,2 ms	145,8 ms	5,0×

C. Inserimento nel diario e persistenza dei dati sensoriali

1) *Rappresentazione geospaziale e query di distanza*: Tutti i dati geospaziali del sistema – i punti GPS, il percorso di ogni segmento, i centroidi delle visite e dei luoghi abituali – sono salvati nel database non come geometria piana, ma con il tipo *geography* di PostGIS (coordinate WGS84, SRID 4326). Questa scelta permette a PostGIS di calcolare le operazioni geospaziali tenendo conto della curvatura della Terra, invece di trattare latitudine e longitudine come coordinate su un piano: in particolare, la lunghezza di un percorso salvato come *LineString* può essere ottenuta con una singola funzione (*ST_Length* su *geografia*), che restituisce direttamente una distanza geodetica in metri, senza che l'applicazione debba implementare a mano formule di distanza tra coordinate. Questa funzione viene usata ripetutamente nel sistema: per calcolare la distanza di ogni segmento di movimento appena costruito, per confrontare la lunghezza di un percorso con quella della sua versione *privacy-aware* (Sezione III), e più in generale ogni volta che serve una misura di distanza reale a partire da una geometria salvata.

2) *Segmentazione del diario*: Nella prima fase di segmentazione del diario, il sistema usa le transizioni della macchina a stati (Sezione III) per individuare macro-intervalli STOP/MOVE: gli intervalli STOP diventano segmenti di sosta senza bisogno di ulteriore elaborazione, poiché la macchina a stati è già la fonte di verità per la sosta. Gli intervalli MOVE, al contrario, vengono affinati in un secondo passo usando le etichette prodotte dal modulo HAR sulle finestre sensoriali che ricadono in quell'intervallo.

Questo affinamento applica due correzioni, a due scale temporali diverse, per distinguere il rumore del classificatore da vere pause nel movimento. La prima corregge cambi di etichetta isolati e brevi: se un gruppo di finestre con un'etichetta diversa è compreso tra due gruppi con la stessa etichetta e dura meno di 60s, viene riassegnato all'etichetta circostante, assumendo che sia un errore di classificazione piuttosto che un cambio di attività reale. La seconda distingue le soste vere da quelle spurie all'interno di un movimento, ed è motivata da un limite intrinseco della macchina a stati stessa: per dichiarare una sosta, la macchina richiede almeno 120s di evidenza di quiete sostenuta (Sezione III). Durante quei 120s, però, il dispositivo continua comunque a raccogliere e classificare finestre sensoriali, ed è molto probabile che il modulo HAR le etichetti correttamente come *IDLE*, perché l'utente è di fatto già fermo – anche se il macro-segmento costruito dalla macchina a stati etichetta ancora quell'intervallo come *movement*, non avendo essa stessa completato la

propria isteresi. Per questo, un gruppo di finestre classificate *IDLE* che dura abbastanza viene riconosciuto come sosta reale e diventa una *pausa virtuale*; un *IDLE* più breve viene invece assorbito nell'etichetta di movimento circostante, trattato come rumore del classificatore. I gruppi risultanti, se adiacenti nel tempo e con la stessa etichetta, vengono infine uniti in un unico segmento (Fig. 5).

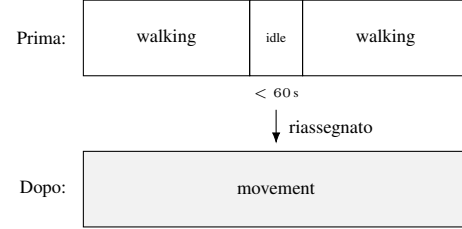


Fig. 5. Correzione di un cambio di etichetta isolato e breve (durata < 60s): il gruppo centrale viene riassegnato all'etichetta circostante.

La distinzione tra pausa virtuale e sosta vera resta interna alla fase di segmentazione: nel diario finale mostrato all'utente le due vengono trattate allo stesso modo, e unite in un'unica sosta (Fig. 6).

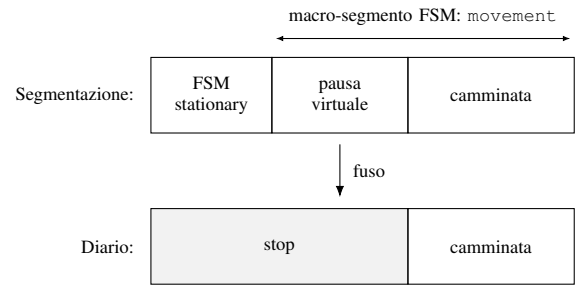


Fig. 6. Una pausa virtuale rilevata dentro un macro-segmento *movement* viene unita, nel diario finale, alla sosta *stationary* adiacente decisa dalla FSM.

3) *Persistenza dei dati sensoriali grezzi*: Una volta completata la costruzione del diario (fasi 1 e 2 della pipeline, Sezione II-D), la terza fase inserisce i dati sensoriali grezzi caricati dal dispositivo nel database delle serie temporali. Qui ogni singolo campione di accelerometro e giroscopio diventa una riga a sé.

L'inserimento avviene tramite *bulk_create* a blocchi di 5.000 righe. Il vantaggio di questa rappresentazione riga per campione si osserva nell'utilizzo analitico che se ne fa.

4) *Utilizzo analitico dei dati sensoriali grezzi*: Inserire i dati grezzi riga per campione in un database rende possibile interrogarli direttamente con query SQL. Il sistema sfrutta questa possibilità per effettuare due analisi visibili dal pannello admin.

La prima query individua eventuali buchi anomali nella registrazione di un singolo viaggio: va a confrontare ogni singolo campione di accelerometro e giroscopio con il precedente all'interno dello stesso segmento di movimento, segnalando intervalli temporali superiori a 10 ms (considerando che i campioni sono acquisiti a una frequenza di 100 Hz).

La seconda query calcola, per ciascuna modalità di attività riconosciuta, la media e la deviazione standard delle sei componenti di accelerometro e giroscopio, considerando tutti i viaggi dell'utente.

D. Data mining dei luoghi abituali

L'individuazione dei luoghi abituali avviene in due fasi successive: prima un rilevamento delle soste (*stay detection*) sui punti GPS grezzi, poi un raggruppamento spaziale di queste soste in luoghi ricorrenti tramite DBSCAN. La prima fase è necessaria perché DBSCAN, di per sé, raggruppa i punti solo in base alla vicinanza spaziale e non tiene conto della dimensione temporale: da solo non può distinguere punti densi perché l'utente è rimasto fermo a lungo in un unico luogo da punti densi perché l'utente vi è tornato più volte in giorni diversi.

La prima fase scorre tutti i punti GPS di un utente in ordine temporale, scartando quelli con precisione peggiore di 60 m, e accumula un cluster di punti consecutivi finché ciascun nuovo punto resta entro 75 m dal centroide corrente del cluster e il tempo trascorso dall'ultimo punto non supera i 90 minuti; al primo punto troppo lontano o dopo un buco temporale troppo lungo, il cluster viene chiuso e se ne inizia uno nuovo. Un cluster diventa una *visita* candidata solo se contiene almeno 3 punti e copre almeno 5 minuti, condizioni che scartano soste troppo brevi.

Le visite individuate vengono poi raggruppate spazialmente con DBSCAN. Per ogni cluster risultante vengono calcolati il centroide, il raggio (la distanza massima tra il centroide e una visita del cluster) e il numero di giorni distinti in cui è stata osservata una visita. Un cluster di punti individuato dal DBSCAN viene promosso a possibile luogo abituale solo se le visite coprono almeno 2 giorni distinti, ed è promosso automaticamente a confermato quando ne raggiunge almeno 4, altrimenti resta in attesa di revisione manuale da parte dell'utente tramite la dashboard. La Fig. 7 riassume l'intera pipeline, dai punti GPS grezzi al ciclo di vita del luogo abituale.

I luoghi confermati vengono infine usati in lettura per arricchire il diario: a ogni segmento di sosta (reale o pausa virtuale) viene associato, se esiste, il luogo confermato più vicino al suo centroide GPS.

Questa dipendenza in lettura ha una conseguenza diretta su come il diario privato viene servito: non esiste, in senso stretto, una versione definitiva e già pronta del diario da restituire, perché il suo contenuto dipende anche dai luoghi abituali dell'utente, che possono cambiare nel tempo indipendentemente dal viaggio stesso – ad esempio quando l'utente conferma, rifiuta o rietichetta un luogo dalla dashboard. Per questo il diario privato è tenuto in una cache (Redis), ma la chiave di cache include, oltre all'identificativo del viaggio, un contatore di versione dei luoghi dell'utente: ogni revisione di un luogo incrementa questo contatore, e la cache esistente diventa automaticamente irraggiungibile con la nuova chiave, senza bisogno di invalidarla esplicitamente. La richiesta

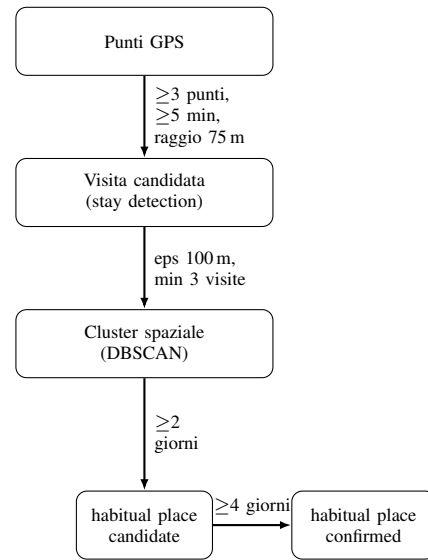


Fig. 7. Pipeline di individuazione dei luoghi abituali: dai punti GPS grezzi alla stay detection, al raggruppamento DBSCAN, fino al ciclo di vita candidate/confirmed del luogo.

successiva del diario trova quindi la cache mancante e lo ricostruisce da zero, con le associazioni ai luoghi aggiornate.

E. Modulo di privacy

Delle tre viste del diario (*precise*, *approximate*, *aggregated*, Sezione II), solo le ultime due applicano davvero una protezione: la vista *precise* non è sottoposta ad alcuna trasformazione ed espone le coordinate GPS esattamente come raccolte, per cui non è un vero livello di privacy ma la vista personale, non protetta, dell'utente sui propri dati. Le viste *approximate* e *aggregated* sono invece ottenute con una tecnica di generalizzazione spaziale a griglia: ogni coordinata GPS viene proiettata in Web Mercator (EPSG:3857) e riposizionata al centro della cella di una griglia di lato fisso a cui appartiene – 150 m per la vista *approximate*, 400 m per *aggregated* – prima di essere riproiettata in coordinate geografiche, come mostrato in Fig. 8.

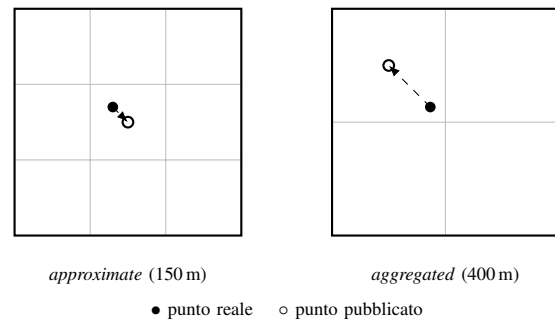


Fig. 8. Generalizzazione spaziale a griglia: lo stesso punto reale viene pubblicato al centro della cella a cui appartiene; una cella più grande (*aggregated*) sposta il punto pubblicato più lontano da quello reale.

La Fig. 9 mostra lo stesso meccanismo su un viaggio reale: la traccia originale (linea continua) e la corrispondente

vista *approximate* (linea tratteggiata), ottenuta sostituendo ogni punto con il centro della sua cella e collegando i centri consecutivi con segmenti retti.

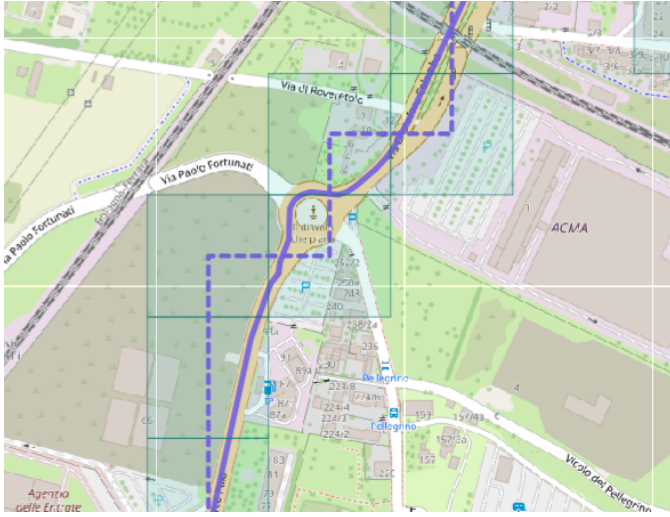


Fig. 9. Esempio su dati reali di un viaggio in auto: la linea continua è il percorso realmente registrato; la linea tratteggiata è la vista *approximate* (griglia da 150m) dello stesso percorso.

Per valutare quantitativamente l'effetto di questa trasformazione, vengono calcolate due metriche per ciascuna traccia. La *Privacy Perturbation* è la distanza media e massima, calcolata con la formula di haversine, tra ciascun punto originale e il corrispondente punto pubblicato dopo l'approssimazione a griglia. La *Quality of Service* è invece misurata come errore relativo tra la lunghezza geodetica del percorso originale e quella del percorso pubblicato, ed esprime quanto la generalizzazione spaziale distorce la distanza realmente percorsa. Queste due metriche, calcolate per i tre livelli di privacy, sono quelle riportate nella valutazione del compromesso privacy/utilità in Sezione IV.

1) Costruzione del diario in base al livello di privacy:

I tre livelli di privacy non si limitano ad approssimare le sole coordinate: incidono sulla costruzione dell'intero diario in modo crescente. Nella vista *precise* nessun dato viene trasformato: ogni segmento mostra l'orario esatto di inizio e fine, il percorso GPS realmente registrato e, di conseguenza, la distanza realmente percorsa; per una sosta, viene mostrato il nome vero del luogo abituale a cui è stata associata, se esiste.

Nella vista *approximate*, gli orari di inizio e fine di ogni segmento vengono arrotondati a un multiplo di 5 minuti (per difetto l'inizio, per eccesso la fine), e il percorso mostrato non è più quello realmente registrato ma quello ottenuto dopo l'approssimazione a griglia descritta sopra: la distanza del segmento viene quindi ricalcolata su questo percorso approssimato, non su quello vero. Il nome del luogo associato a una sosta, invece, resta visibile come nella vista precisa: poiché la posizione è già protetta dall'approssimazione a griglia delle coordinate, mostrare anche il nome del luogo non aggiunge un rischio significativo ulteriore.

La vista *aggregated* applica le stesse trasformazioni di orario e percorso della vista approssimata, ma con una griglia più larga (400m anziché 150m), e in più nasconde anche l'identità dei luoghi: al posto del nome del luogo abituale mostra solo la sua categoria generica o un'etichetta neutra se la sosta non è associata a nessun luogo confermato.

IV. RISULTATI

A. Compromesso privacy/utilità

Le metriche di Privacy Perturbation e Quality of Service definite in Sezione III sono state calcolate sull'insieme dei viaggi realmente registrati nel database, segmento per segmento: separatamente per i segmenti di movimento, raggruppati per modalità di attività riconosciuta, e per i segmenti di sosta, per cui la perturbazione è calcolata sul centroide GPS della sosta anziché sulla lunghezza di un percorso. La Tabella III riporta i risultati.

TABLE III
PRIVACY PERTURBATION MEDIA E QUALITY OF SERVICE (ERRORE RELATIVO DI DISTANZA) PER MODALITÀ, SUI DUE LIVELLI PROTETTI

Modalità	n	Perturbation (m)		QoS (err. rel.)	
		approx.	aggr.	approx.	aggr.
Veicolo	38	37,3	100,3	0,64	0,61
A piedi	56	41,9	93,0	0,93	1,06
Bicicletta	3	44,8	82,3	0,50	1,28
Sosta	51	35,7	97,5	–	–

La Privacy Perturbation cresce con la dimensione della cella, come atteso: passando da *approximate* (150 m) ad *aggregated* (400 m) la distanza media tra punto reale e punto pubblicato più che raddoppia in tutte le modalità, comprese le soste. La Quality of Service, invece, non peggiora sempre in modo monotono con l'aggregazione: per i segmenti in auto l'errore relativo di distanza è leggermente più basso con celle più grandi (0,61 contro 0,64) che con celle più piccole. Il campione di segmenti in bicicletta è troppo piccolo ($n=3$) per trarne conclusioni robuste, ed è riportato solo per completezza.

B. Latenza della pipeline di ingestione

Per misurare il tempo impiegato dalle diverse fasi della pipeline di caricamento ed elaborazione di un viaggio è stato eseguito un test di carico con un client sintetico che riproduce il comportamento dell'applicazione mobile reale (Sezione III). Sono stati simulati 25 utenti che caricano contemporaneamente viaggi di circa un'ora: poiché un utente non può avere più di un viaggio in corso alla volta (Sezione III), ciascuno ha caricato i propri viaggi in sequenza, uno alla volta, per un totale di 10 viaggi a testa (250 viaggi complessivi). La fase di upload raw è stata cronometrata lato client; le altre quattro fasi, lato backend. Il test è stato eseguito su un ambiente locale equivalente a quello di produzione (stessi container Docker), non sull'hardware di produzione stesso: i valori assoluti vanno quindi letti come indicativi, mentre le proporzioni relative tra le fasi restano informative.

La fase nettamente più onerosa è l'inserimento in batch dei dati sensoriali grezzi (media 29,68 s, fino a quasi 1 minuto

TABLE IV
LATENZA MEDIA DELLE FASI DELLA PIPELINE DI INGESTIONE (250
VIAGGI, 25 UTENTI CONCORRENTI)

Fase	Media	Dev.std	Min	Max
Core (GPS+transizioni)	2,02 s	2,61 s	0,25 s	15,83 s
Upload raw (client)	3,04 s	2,56 s	0,43 s	28,59 s
HAR + diario	4,62 s	1,50 s	1,99 s	11,06 s
Batch insert raw	29,68 s	8,235 s	20,39 s	53,97 s
Data mining luoghi	0,97 s	0,64 s	0,27 s	2,74 s

nel caso peggiore), dovuto al volume di righe scritte in `RawSensorReading` – una per ogni campione, quindi fino a 360.000 righe per un singolo viaggio di un’ora (720 finestre da 500 campioni). L’inserimento usa già `bulk_create` a gruppi di 5000 righe (72 batch per un viaggio di un’ora), quindi il costo non è dovuto a query singole per riga, ma al volume assoluto di dati scritti a questa granularità, aggravato dalla contesa sul database sotto 25 utenti concorrenti. Va però notato che il batch insert dei dati grezzi non è una fase percepita dall’utente: il diario del viaggio è già disponibile non appena termina la fase HAR, mentre la persistenza dei dati grezzi prosegue in background. La latenza effettivamente percepita dall’utente – core, upload raw e HAR/diario – resta quindi complessivamente bassa, dell’ordine di pochi secondi.

Per correttezza va infine segnalato che la velocità della fase HAR è in parte dovuta al fatto che il modello viene caricato in memoria una sola volta all’avvio di ciascun processo worker Celery (`warm_har_model_on_worker_start`, agganciato al segnale `worker_process_init`), anziché ad ogni singola richiesta di inferenza. Questo rende l’inferenza rapida a runtime, ma ha come contropartita un utilizzo di RAM elevato per ciascun worker, dato il peso del modello tenuto costantemente in memoria.

Questo suggerisce due direzioni per un’eventuale scalatura del sistema oltre il semplice aumento del numero di worker. La prima riguarda proprio il modello HAR: passare da una copia caricata in RAM per ciascun worker a un’unica istanza condivisa, interrogata dai worker come servizio separato, ridurrebbe l’occupazione di memoria aggregata a scapito di una latenza leggermente più alta sulla singola richiesta di inferenza – un compromesso favorevole quando l’obiettivo è aumentare il numero di worker eseguibili in parallelo sulla stessa macchina. La seconda riguarda il database: durante il test di carico è stato il componente più sollecitato in assoluto (più dei worker Celery stessi), per cui scalare orizzontalmente il sistema richiederebbe anche di replicare il database e introdurre una logica di sincronizzazione tra le repliche, oltre alla sola duplicazione dei worker applicativi.

CONCLUSIONE

In questo lavoro ho presentato Mobility Diary, un sistema context-aware per la costruzione di un diario personale di mobilità, composto da un’applicazione mobile, un back-end che ne gestisce l’elaborazione asincrona e una dashboard amministrativa. Il sistema integra una macchina a stati finiti per il rilevamento del movimento, un modulo di riconoscimento

dell’attività umana basato su CNN1D e GRU, e meccanismi di generalizzazione spaziale per proteggere la posizione dell’utente su tre livelli di privacy.

Il modulo HAR raggiunge un’accuratezza dell’84,88% sul validation set ufficiale dopo l’ottimizzazione per l’inferenza in produzione. Sul fronte privacy, l’analisi condotta sui viaggi realmente registrati mostra che la Privacy Perturbation cresce in modo prevedibile con la dimensione della cella di generalizzazione, mentre l’effetto sull’utilità del dato varia a seconda della modalità di movimento: per gli spostamenti in auto, tipicamente lunghi e rettilinei, la Quality of Service resta pressoché stabile anche con la griglia più larga, mentre per la camminata, fatta di percorsi più brevi, l’aggregazione comporta una perdita di utilità più marcata.

Sul fronte delle prestazioni, il test di carico sulla pipeline di ingestione mostra che la latenza effettivamente percepita dall’utente – upload dei dati e classificazione HAR, dopo la quale il diario è già consultabile – resta dell’ordine di pochi secondi, mentre la persistenza dei dati sensoriali grezzi, molto più lenta, avviene in background senza impatto percepito. Sotto carico concorrente il database si conferma il componente più sollecitato, indicando che una futura scalata del sistema richiederebbe la sua replica, oltre a un modello HAR condiviso tra i worker invece che duplicato in RAM su ciascuno.

Il sistema presenta comunque dei limiti. La macchina a stati finiti che rileva il movimento è un’euristica costruita ad hoc su accelerometro e GPS, ed è verosimilmente meno precisa delle API di riconoscimento dell’attività fornite nativamente dal sistema operativo del telefono, che non sono state utilizzate. Il modulo HAR, inoltre, non distingue tra le diverse modalità di trasporto motorizzato – auto, autobus, treno – che vengono tutte classificate in un’unica etichetta `MOVING_VEHICLE`.

Due direzioni di sviluppo futuro appaiono particolarmente rilevanti. La prima è l’introduzione di un processo periodico che elimini dallo storage a oggetti i dati sensoriali grezzi di un viaggio una volta che la sua elaborazione è completata: attualmente questi dati non vengono mai rimossi automaticamente, e restano nello storage indefinitamente a meno che l’utente non cancelli esplicitamente il viaggio. La seconda è un meccanismo con cui l’utente possa correggere le etichette di attività assegnate dal modulo HAR quando errate (ad esempio segnalando che un tratto indicato come cammino era in realtà percorso in auto), e una pipeline che raccolga queste correzioni per aggiornare periodicamente i pesi del modello.